

# Dy3DGS-SLAM: Monocular 3D Gaussian Splatting SLAM for Dynamic Environments

Mingrui Li<sup>1†</sup>, Yiming Zhou<sup>2,3,5†</sup>, Hongxing Zhou<sup>4</sup>, Xinggang Hu<sup>1</sup>, Florian Roemer<sup>3</sup>,  
Hongyu Wang<sup>1\*</sup>, Ahmad Osman<sup>2,3</sup>

**Abstract**—Current Simultaneous Localization and Mapping (SLAM) methods based on Neural Radiance Fields (NeRF) or 3D Gaussian Splatting excel in reconstructing static 3D scenes but struggle with tracking and reconstruction in dynamic environments, such as real-world scenes with moving elements. Existing NeRF-based SLAM approaches addressing dynamic challenges typically rely on RGB-D inputs, with few methods accommodating pure RGB input. To overcome these limitations, we propose Dy3DGS-SLAM, **the first 3D Gaussian Splatting (3DGS) SLAM method for dynamic scenes using monocular RGB input**. To address dynamic interference, we fuse optical flow masks and depth masks through a probabilistic model to obtain a fused dynamic mask. With only a single network iteration, this can constrain tracking scales and refine rendered geometry. Based on the fused dynamic mask, we designed a novel **motion loss to constrain the pose estimation network for tracking**. In mapping, we use the **rendering loss of dynamic pixels, color, and depth to eliminate transient interference and occlusion caused by dynamic objects**. Experimental results demonstrate that Dy3DGS-SLAM achieves state-of-the-art tracking and rendering in dynamic environments, outperforming or matching existing RGB-D methods.

## I. INTRODUCTION

Recently, dense SLAM systems expressed through NeRF [1] or 3DGS [2] have attracted significant attention. These systems have achieved photo-realistic rendering results in static scenes and are gradually expanding towards large-scale or challenging scenarios. However, a practical issue for SLAM systems is evident: the real world contains a large number of dynamic objects, and current NeRF or 3DGS-based SLAM systems [3]–[7] perform poorly in addressing this challenge. Another issue that has gained attention is how to achieve better results without relying on RGB-D sensors and using only monocular RGB input, which is considered a more accessible sensor and a solution with greater potential.

Although some NeRF-based methods have attempted to address dynamic objects, such as DN-SLAM [8], DDN-SLAM [9], NID-SLAM [10], and RoDyn-SLAM [11], they often rely on predefined dynamic priors or heavily depend on depth priors to determine dynamic object masks, making them unsuitable for environments with only monocular RGB input. Furthermore, due to the limitations of NeRF [1] representation, there are constraints on rendering accuracy, often resulting in severe rendering artifacts. 3DGS-based SLAM systems, such as SplatTAM [12], Photo-SLAM [13], and MonoGS [14], perform well in static environments, but they tend to encounter tracking failures and mapping errors in dynamic scenes.

Therefore, we propose Dy3DGS-SLAM, the first RGB-only 3DGS-SLAM system designed for dynamic environments. We utilize optical flow to obtain dynamic masks without relying on predefined moving objects, though these masks can be noisy in regions with uniform textures or fast motion. To address this, we incorporate monocular depth estimation, providing complementary spatial cues, especially for occlusions and depth discontinuities. We then propose a depth-regularized mask fusion strategy that combines the strengths of both modalities, mitigating individual limitations and producing more precise, robust dynamic masks.

For tracking, we incorporate the estimated depth and fused mask into the motion loss, effectively recovering scale and pose in the pose estimation network, resulting in more accurate tracking outcomes. In terms of rendering, to address transient interference and occlusion, we penalize dynamic Gaussians based on the color and depth of dynamic pixels. Compared to baseline methods, our approach significantly reduces rendering artifacts and greatly improves geometric accuracy. In summary, our method has the following contributions:

- We propose Dy3DGS-SLAM, the first RGB-only 3DGS-SLAM system for dynamic environments, capable of robust tracking and high-fidelity reconstruction in dynamic environments.
- We propose a mask fusion method that accurately covers dynamic objects by combining motion cues from optical flow with geometric consistency from depth estimation. Based on the fused mask, we introduce novel motion and rendering losses to effectively mitigate dynamic object interference in tracking and rendering.
- Our results on three real-world datasets demonstrate that our method achieves better tracking and rendering performance compared to baseline methods.

<sup>†</sup> Both authors contributed equally to the paper.

<sup>\*</sup> Corresponding author.

<sup>1</sup> Mingrui Li, Xinggang Hu and Hongyu Wang are with the School of Information and Communication Engineering, Dalian University of Technology, Dalian, 116024, China 2905450254@mail.dlut.edu.cn

<sup>2</sup> Yiming Zhou and Ahmad Osman are with the School of Engineering Sciences, Saarland University of Applied Sciences, Saarbruecken, 66117, Germany yiming.zhou@htwsaar.de, ahmad.osman@htwsaar.de

<sup>3</sup> Florian Roemer, Yiming Zhou and Ahmad Osman are with Fraunhofer Institute for Nondestructive Testing, Saarbruecken, 66123, Germany Florian.Roemer@izfp.fraunhofer.de

<sup>4</sup> Hongxing Zhou is with the College of Information Science and Technology, Beijing University of Chemical Technology, Beijing, 100029, China hongxingzhou@mail.buct.edu.cn

<sup>5</sup> Yiming Zhou is with the Faculty of Science and Engineering, Laval University, Quebec, GEV 0A6, Canada yiming.zhou.1@laval.ca

## II. RELATED WORK

Vision-based SLAM systems are essential technologies for addressing mapping challenges in robotics and scene reconstruction in VR/AR applications [15]–[17]. In real-world scenarios, dynamic objects pose a significant challenge to visual SLAM systems. There has been extensive exploration in the traditional visual SLAM field to tackle the interference caused by dynamic objects. Recently, deep learning-based methods have gained attention, mainly falling into two categories. One type focuses on semantic prior-based segmentation, represented by systems such as DS-SLAM [18], OVD-SLAM [19], and SG-SLAM [20]. These methods utilize deep learning frameworks to recognize semantic information and then use epipolar geometry or depth constraints to identify dynamic points and remove them. The second type relies on optical flow estimation methods, such as FlowFusion [21], DeflowSLAM [22], and DytanVO [23], using deep learning-based optical flow estimation frameworks to estimate camera poses. However, these methods often lack dense and stable reconstruction and fail to accurately recover depth or obtain precise dynamic object masks under monocular conditions.

With the advent of NeRF and 3DGS showing high-fidelity reconstruction and fast rendering capabilities in 3D reconstruction, there has been a growing interest in RGB-D SLAM systems. However, these systems often perform poorly in real-world dynamic environments. Some NeRF-based RGB-D SLAM systems have explored this problem. For instance, DN-SLAM [8] uses optical flow estimation to remove dynamic points and employs the Instant NGP-based [24] rendering framework for view synthesis, although it struggles with artifacts. NID-SLAM [10] leverages an optical flow estimation system to obtain dynamic masks and completes background reconstruction, but its tracking accuracy is limited, and the rendering process lacks sufficient constraints. Rodyn-SLAM [11] uses a sliding window optical flow estimation method to acquire motion masks and proposes specific rendering losses, but it heavily depends on prior depth information provided by the sensors. Our approach utilizes the advantages of 3DGS for representation while addressing the reliance on depth sensors through a depth estimation system. By using optical flow estimation, we generate more accurate motion masks, enabling the reconstruction of static scenes effectively.

## III. METHOD

Our system pipeline is shown in Fig. 1. In Section III-A, we address the problem of fusing the dynamic mask obtained from optical flow with the depth map estimated from monocular input, resulting in an accurate dynamic fusion mask. In Section III-B, we propose the motion estimation network and introduce a motion loss incorporating depth estimation, enabling the network to iteratively refine accurate camera poses. In Section III-C, we penalize the Gaussians corresponding to pixels labeled as dynamic and apply an additional rendering loss based on monocular depth to optimize the scene details. Finally, we synthesize a static scene using multi-view consistency.

### A. Dynamic Mask Fusion

Our tracking network includes an optical flow estimation module, a depth estimation module, a mask fusion method, and a pose estimation module. Assume that we have two consecutive undistorted images,  $I_t$  and  $I_{t+1}$ , as input, and the output is the relative camera motion  $E = (R|T)$ , where  $T \in \mathbb{R}^3$  represents the 3D translation and  $R \in \text{SO}(3)$  represents the 3D rotation.

To detect optical flow anomalies caused by dynamic objects, we employ a lightweight U-Net [25] motion segmentation network. It takes the original input frames as input and binarizes the result, setting all optical flow within the mask area to zero, and finally obtains the corresponding optical flow values  $F$  and the optical flow mask  $F_m$ . However, this estimation is relatively coarse and static areas are easily mis-segmented, affecting the accuracy of camera pose estimation. Therefore, we introduce additional depth supervision using the DepthanythingV2 [26] estimation network to provide estimated depth, obtaining the corresponding depth mask  $D_m$ .

Although we obtained dynamic masks generated by the optical flow estimation network, there are instances of incorrect mask estimation. Therefore, we need to use the depth mask to correct the optical flow mask. We combine depth and optical flow information using conditional probability to obtain a more accurate fusion mask  $\hat{M}$  for determining dynamic regions.

First, we aggregate optical flow pixels to separate multiple potential moving objects. We use the K-means clustering algorithm to segment the motion pixels. Let the set of pixels in the motion region be  $P_{\text{dynamic}} = \{p \mid M(p) = 1\}$ . We divide these motion pixels into  $k$  clusters, with each cluster representing an independent dynamic object. The goal of clustering is to minimize the sum of squared pixel distances within each cluster. The clustering objective function is defined as follows:

$$\min_{\mu_1, \mu_2, \dots, \mu_k} \sum_{i=1}^k \sum_{p \in C_i} \|p - \mu_i\|^2, \quad (1)$$

where  $C_i$  represents the set of pixels assigned to the  $i$ -th cluster, and  $\mu_i$  is the center of the  $i$ -th cluster, defined as:

$$\mu_i = \frac{1}{|C_i|} \sum_{p \in C_i} p. \quad (2)$$

Through the clustering result, we can separate moving objects and ensure the correctness of the subsequent fusion mask process. Finally, we obtain the set of moving objects  $N = \{N_1, N_2, \dots, N_k\}$ , where  $N_i$  represents the pixel set of the  $i$ -th moving object. We perform a separate depth probability search for each moving object to achieve mask fusion. Since the depth map and optical flow mask are independent, we propose a Bayesian model for estimating the probability:

$$P(D_m, F_m \mid M(p)) = P(D_m \mid M(p)) \cdot P(F_m \mid M(p)). \quad (3)$$

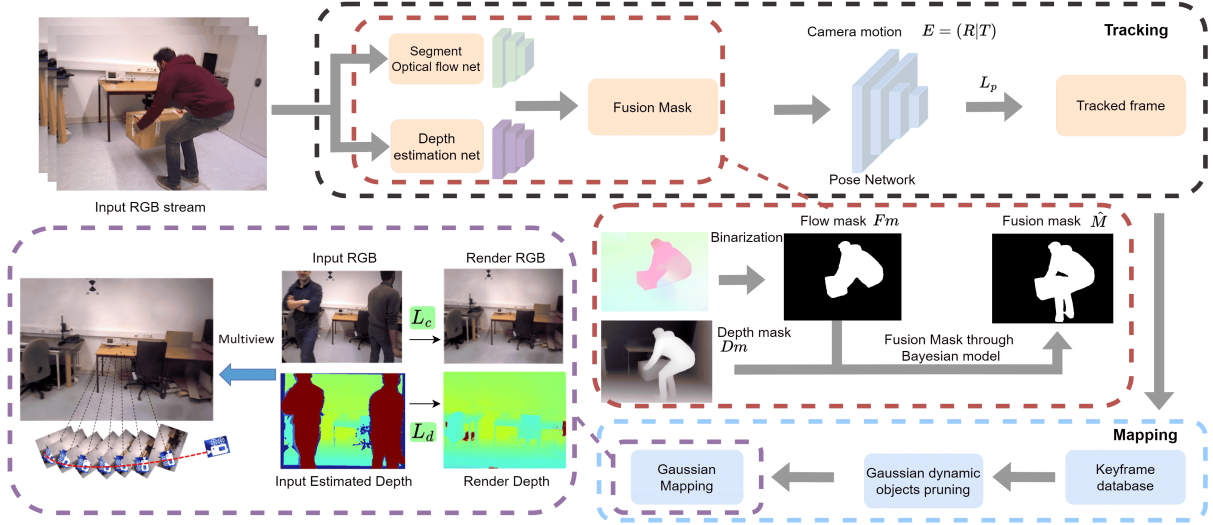


Fig. 1: **Pipeline of Our Network:** Our system workflow consists of two main threads: tracking and mapping. In the tracking thread, we use a segmentation optical flow network and a depth estimation network to generate the estimated motion optical flow mask and depth map mask. By applying a conditional probability approach, we create a fused mask  $\hat{M}$ . This fused mask is subsequently input into the pose estimation network to determine the estimated pose. In the mapping thread, we utilize the fused mask  $\hat{M}$  along with keyframes to construct the map. We impose color and depth penalties on the Gaussians corresponding to the moving pixels identified by the fused mask, which ultimately results in a multi-view rendering outcome.

We binarize the mask  $M(p)$  by setting it to 1, where  $P(M(p) = 1 | D_m, F_m)$  represents the posterior probability that a pixel belongs to a moving object.  $P(D_m | M(p) = 1)$  is the likelihood of observing the depth map  $D_m$  given that the pixel is part of a moving object, and  $P(F_m | M(p) = 1)$  is the likelihood of observing the optical flow mask  $F_m$  under the same assumption.  $P(M(p) = 1)$  denotes the prior probability that a pixel is part of a moving object.

Finally, by combining all known information and calculating the posterior probability for each pixel, a new motion mask for each pixel is obtained:

$$\hat{M}_i = \begin{cases} 1 & \text{if } P(M(p) = 1 | D_m, F_m) > T \\ 0 & \text{otherwise} \end{cases}, \quad (4)$$

where  $T$  is the pixel probability threshold, set to 0.95 in our method.

The final fused mask for all moving objects is given by  $\hat{M} = M(N_1) \cup M(N_2) \cup \dots \cup M(N_i)$ . Our method does not require additional network iterations and is more generalized, allowing it to handle multiple moving objects simultaneously without the need for scene-specific parameter adjustments.

### B. Tracking for Monocular Dynamic Scenes

Unlike other visual odometry systems [27], our systems can provide accurate pixel depth constraints due to the use of estimated depth information. This is significant in addressing pose estimation errors caused by depth ambiguity. We obtain the static depth mask  $M_{ds} = \{p \in \hat{M} | M(p) = 0\}$  through the fusion of dynamic masks. The static depth mask is applied to the optical flow map, and the corresponding scale factor  $S_n$  is fused to obtain the formula:

$$\tilde{F} = F \cdot M_{ds} \cdot S_n. \quad (5)$$

In this formula,  $F$  is the optical flow map,  $M_{ds}$  filters the residual mask of untrustworthy dynamic regions, and  $S_n$  provides accurate scale information for the remaining static regions. Through this method, the network can apply more reliable depth information to the static areas, avoiding interference from dynamic regions while correcting scale errors in monocular estimation.

We fuse the acquired static mask  $M_{ds}$  with the optical flow mask  $\tilde{F}$  and input them into the network to update the pose. To achieve a more accurate iterative pose estimation process, we introduce a camera motion loss  $\mathcal{L}_M$ , adjusting the estimated pose distance from the ground truth. The loss function with the introduced scale constraint is expressed as:

$$\mathcal{L}_M = \frac{\hat{T}}{\max(|\hat{T} \cdot S_n|, \varepsilon)} - \frac{T}{\max(|T \cdot S_n|, \varepsilon)} + (\hat{R} - R) \cdot M_{ds}, \quad (6)$$

where  $S_n$  is used to adjust the scale of the translation vector to align it with the true scale value. We perform pose updates within a pose estimation network based on ResNet50 [28], following the training design in TartanVO [27]. The network jointly optimizes optical flow loss  $\mathcal{L}_O$ , motion segmentation loss  $\mathcal{L}_U$ , and camera motion loss  $\mathcal{L}_M$ , which incorporates depth masking and scale constraints. DytanVO [23] improves camera pose estimation and dynamic mask segmentation through three iterations, but the final results still have limitations. In contrast, our method requires only a single iteration without incurring additional computational costs. The comprehensive tracking loss function is formulated as:

$$\mathcal{L}_P = \lambda_1 \mathcal{L}_O + \lambda_2 \mathcal{L}_U + \mathcal{L}_M, \quad (7)$$

where  $\lambda_1$  and  $\lambda_2$  are weights that control the different loss terms, ensuring that the network can balance the tasks

of optical flow, motion segmentation, and pose estimation during training.

For more accurate pose estimation, we generate a keyframe every 10 frames and create a keyframe group consisting of at least 4 keyframes, applying local Bundle Adjustment optimization to correct accumulated errors.

### C. Gaussian Rendering for Monocular Dynamic Scenes

In 3DGS [2], an explicit point-based scene representation is optimized. Each 3D Gaussian is parameterized by a set of 3D attributes, including position, opacity, scale, and rotation. The Gaussian ellipsoid is characterized by a full 3D covariance matrix  $\Sigma$ , which is defined (normalized) in world space. The Gaussian function is defined as:

$$g(\mathbf{x}) = o \exp\left(-\frac{1}{2}\mathbf{x}^T \Sigma^{-1} \mathbf{x}\right), \Sigma = \mathbf{R} \mathbf{S} \mathbf{S}^T \mathbf{R}^T, \quad (8)$$

where  $\Sigma$  is the covariance matrix,  $o \in [0, 1]$  represents the opacity value,  $S$  is the scale matrix, and  $R$  is the rotation matrix.

We use 3D Gaussian ellipsoids to render 2D images through splatting techniques, as described in [29], [30]. In the camera coordinate system, the covariance matrix  $\Sigma'$  is formulated as:

$$\Sigma' = \mathbf{J} \mathbf{W} \Sigma \mathbf{W}^T \mathbf{J}^T, \quad (9)$$

where  $\mathbf{W}$  represents the viewing direction, and  $\mathbf{J}$  is the Jacobian matrix of the affine approximation of the projection transformation. For each pixel, the color and opacity of all Gaussian ellipsoids are computed and blended using the following formula:

$$C = \sum_{i \in N} c_i g_i \prod_{j=1}^{i-1} (1 - g_j), \quad (10)$$

where  $c_i$  represents the color of the  $i$ -th Gaussian ellipsoid. Additionally, we propose a similar formula for depth rendering:

$$D = \sum_{i=1}^n d_i g_i \prod_{j=1}^{i-1} (1 - g_j), \quad (11)$$

where  $d_i$  is the z-axis depth of the center of the  $i$ -th 3D Gaussian.

Since our method is based on keyframe multi-view rendering, each Gaussian  $g_i$  is associated with a keyframe that anchors it to the map  $G$ . For the Gaussians produced by pixels marked as dynamic, we set their **depth to infinity to perform pruning**. However, this may cause artifacts that are hard to remove, so we apply photometric loss  $L_c$  and depth loss  $L_d$  to the pixel masking process to eliminate the artifacts' impact.

The new photometric loss is:

$$L_c = \lambda_d \cdot \frac{N_d}{N_{pi}} |C_k - C_k^{gt}| + \lambda_s \cdot \frac{N_{pi} - N_d}{N_{pi}} |C_k - C_k^{gt}|, \quad (12)$$

where  $\lambda_d$  is the penalty factor for dynamic pixel masks,  $\lambda_s$  is the penalty factor for static pixel masks,  $N_{pi}$  is the number

of pixels in each keyframe, and  $N_d$  represents the number of pixels corresponding to the dynamic mask.

The new depth loss is:

$$L_d = \lambda_t \cdot \frac{D_d}{D_{pi}} |D_k - D_k^e| + \lambda_m \cdot \frac{D_{pi} - D_d}{D_{pi}} |D_k - D_k^e|, \quad (13)$$

where  $\lambda_t$  is the penalty factor for dynamic depth masks,  $\lambda_m$  is the penalty factor for static depth masks,  $D_{pi}$  is the number of pixels corresponding to the estimated depth in each keyframe,  $D_d$  represents the depth corresponding to the dynamic mask, and  $D_k^e$  represents the depth generated by monocular estimation.

The final rendering loss function  $L_G$  is:

$$L_G = L_c + \lambda \cdot L_d, \quad (14)$$

where  $\lambda$  is a hyperparameter set to 1.

## IV. EXPERIMENTAL RESULTS

### A. Experimental Details and Metrics

**Datasets and Implementation details.** We evaluated our method on three public datasets from the real world: the TUM RGB-D dataset [31], AirDOS-Shibuya dataset [32] and the BONN RGB-D dynamic dataset [33], all of which capture real indoor environments. We conducted our SLAM experiments on a desktop equipped with a single RTX 3090 Ti GPU. We present results from our multiprocess implementation designed for real-time applications. Consistent with the 3DGS framework, time-critical rasterisation and gradient computation are implemented using CUDA.

**Metrics and Baseline Methods.** To evaluate camera tracking accuracy, we report the Root Mean Square Error (RMSE) of the Absolute Trajectory Error (ATE) for keyframes. For runtime performance and network iteration speed, we measure frames per second (FPS) and milliseconds (ms), respectively. GPU usage is assessed in megabytes (MB). We compare our Dy3DGS-SLAM method against traditional dynamic SLAM approaches, such as ORB-SLAM3 [34], Droid-SLAM [35], DynaSLAM [11], DytanVO [23] and ReFusion [33], as well as state-of-the-art NeRF-based methods utilizing RGB-D sensors, including NICE-SLAM [4], ESLAM [36], Co-SLAM [37], and NID-SLAM [10]. Furthermore, we consider SplatAM [12], which is based on 3DGS.

### B. Evaluation on TUM and Bonn RGB-D

**Tracking.** As shown in Table II, we present results for three highly dynamic sequences, one mildly dynamic sequence, and two static sequences from the TUM dataset [31]. Thanks to our proposed dynamic mask fusion method, our system demonstrates advanced tracking performance compared to RGB-D-based methods and is even competitive with traditional SLAM methods. Furthermore, we evaluated the tracking performance on the more complex and challenging BONN dataset [33], as illustrated in Table I. Even in these more complicated and large-scale scenarios, our method achieved superior performance. Our method outperforms all other approaches, with NID-SLAM [10] being the only one



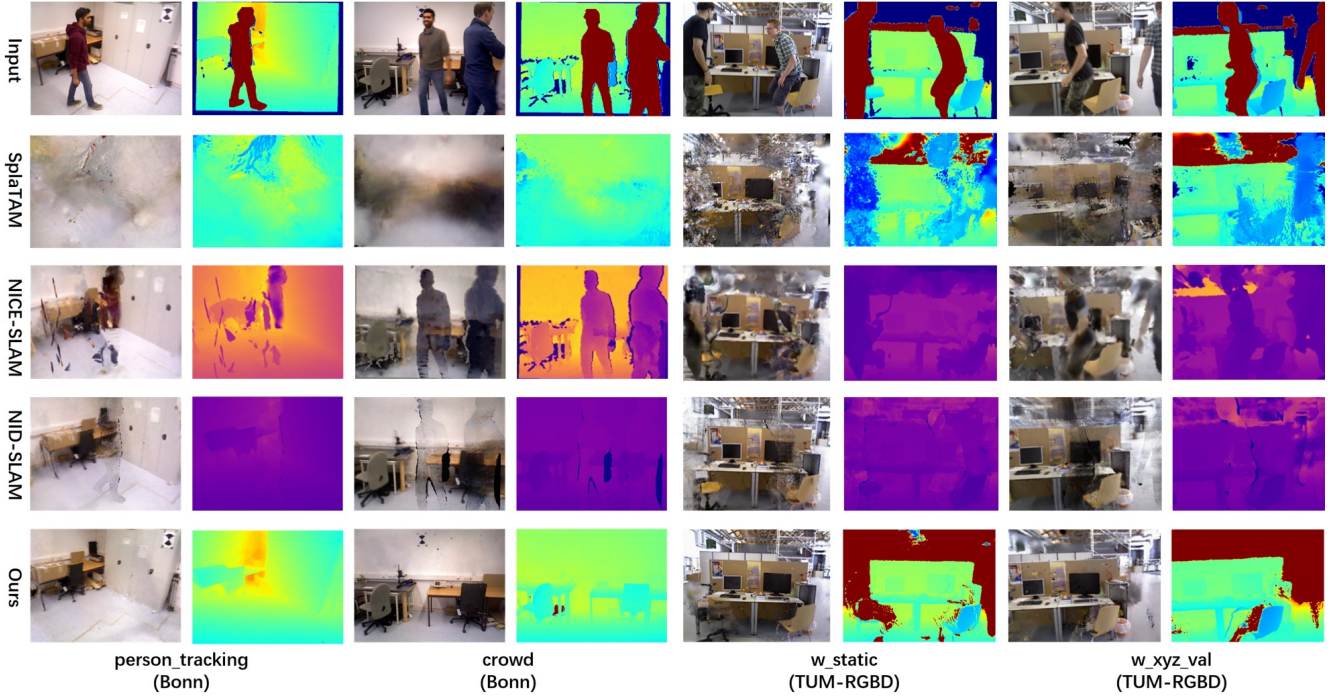


Fig. 2: Visual comparison of the reconstructed meshes on the BONN and TUM RGB-D datasets. Our results are more complete and accurate without the dynamic object floaters.

TABLE I: Tracking performance on *BONN-RGB-D* (ATE RMSE ↓ [cm]). The best results are highlighted as **first**, **second** and **third**.

| Methods                                | ballon     | ballon2    | ps_track   | ps_track2  | mv_box1    | mv_box2    | Avg.       |
|----------------------------------------|------------|------------|------------|------------|------------|------------|------------|
| <i>Traditional SLAM methods</i>        |            |            |            |            |            |            |            |
| ORB-SLAM3 [34]                         | 5.8        | 17.7       | 70.7       | 77.9       | <b>3.1</b> | <b>3.7</b> | 29.8       |
| Droid-SLAM [35]                        | 5.4        | 4.6        | 21.3       | 46.0       | 8.9        | 5.9        | 15.4       |
| DynaSLAM [38]                          | <b>3.0</b> | <b>3.0</b> | <b>6.1</b> | <b>7.8</b> | <b>4.9</b> | <b>3.9</b> | <b>4.8</b> |
| ReFusion [33]                          | 17.5       | 25.4       | 28.9       | 46.3       | 30.2       | 17.9       | 27.7       |
| DytanVO [23]                           | 6.3        | 3.1        | <b>3.5</b> | <b>9.0</b> | 5.5        | 6.1        | <b>5.6</b> |
| <i>NeRF or 3DGS based SLAM methods</i> |            |            |            |            |            |            |            |
| NICE-SLAM [4]                          | 80.3       | 66.8       | 54.9       | 45.3       | 21.2       | 31.9       | 44.1       |
| Co-SLAM [37]                           | 28.8       | 20.6       | 61.0       | 59.1       | 38.3       | 70.0       | 46.3       |
| ESLAM [36]                             | 22.6       | 36.2       | 48.0       | 51.4       | 12.4       | 17.7       | 31.4       |
| NID-SLAM [10]                          | <b>3.7</b> | <b>2.8</b> | 10.0       | 14.7       | 6.9        | 4.2        | 7.1        |
| SplaTAM [12]                           | 231.9      | 126.4      | 27.8       | 53.1       | 63.7       | 73.6       | 96.1       |
| Dy3DGS-SLAM (Ours)                     | <b>4.5</b> | <b>1.9</b> | <b>5.6</b> | <b>6.4</b> | <b>5.2</b> | <b>3.8</b> | <b>4.5</b> |

achieving results close to ours. Additionally, our method demonstrates superior performance compared to traditional methods. This highlights our dynamic mask fusion can effectively remove the dynamic objects and enhance the tracking process.

**Mapping.** To comprehensively evaluate the performance of our proposed system in dynamic scenes, we analyze the results from a qualitative perspective. We compare the rendered images with ground truth poses obtained from the generated Gaussian map, using the same viewpoint as other methods. Four challenging sequences were selected: *crowd* and *person\_tracking* from the BONN dataset, and *f3\_walk\_xyz\_val* and *f3\_walk\_static* from the TUM RGB-D dataset. As shown in Fig. 2, our method shows significant

advantages in geometric and texture details, especially in reducing artifacts. Notably, our approach is based on a monocular system and has been validated on two real-world datasets, demonstrating its capability to accurately record dynamic scenes with just a simple camera. This highlights the potential of our method to effectively track and reconstruct indoor environments, making it a valuable tool for applications where depth sensors may not be available.

## V. ABLATION STUDY

### A. Fusion Mask Strategy Evaluation

To evaluate the effectiveness of the proposed methods in our system, we conduct ablation studies on five scenes from the AirDOS-Shibuya dataset, with all results being

TABLE II: Tracking performance on *TUM-RGB-D* (ATE RMSE ↓ [cm]). “-” denotes the absence of mention. “X” denotes the tracking failures. The best results are highlighted as **first**, **second** and **third**.

| Methods                         | High Dynamic |          |          |          | Low Dynamic |           | Avg. |
|---------------------------------|--------------|----------|----------|----------|-------------|-----------|------|
|                                 | f3/wk_xyz    | f3/wk_hf | f3/wk_st | f3/st_hf | f3/st_xyz   | f1/st_rpy |      |
| Traditional SLAM methods        |              |          |          |          |             |           |      |
| ORB-SLAM3 [34]                  | 28.1         | 30.5     | 2.0      | 2.6      | 2.2         | 2.8       | 11.5 |
| DVO-SLAM [39]                   | 59.7         | 52.9     | 21.2     | 6.2      | 2.1         | 3.0       | 23.2 |
| DynaSLAM [38]                   | 1.7          | 2.6      | 0.7      | 2.8      | 1.6         | 5.1       | 2.7  |
| ReFusion [33]                   | 9.9          | 10.4     | 1.7      | 11.0     | -           | -         | 8.3  |
| DytanVO [23]                    | 8.7          | 9.8      | 9.5      | 14.7     | 12.4        | 12.3      | 11.2 |
| NeRF or 3DGS based SLAM methods |              |          |          |          |             |           |      |
| NICE-SLAM [4]                   | 113.8        | X        | 137.3    | 93.0     | 43.9        | 65.6      | 90.6 |
| Co-SLAM [37]                    | 51.8         | 105.1    | 49.5     | 4.7      | 9.3         | 8.9       | 36.3 |
| ESLAM [36]                      | 45.7         | 60.8     | 93.6     | 3.6      | 8.6         | 9.2       | 34.5 |
| NID-SLAM [10]                   | 6.4          | 7.1      | 6.2      | 10.9     | 7.5         | 8.6       | 7.8  |
| SplaTAM [12]                    | 41.3         | 72.6     | 45.7     | 75.9     | 32.8        | 40.5      | 53.2 |
| Dy3DGS-SLAM (Ours)              | 5.8          | 7.0      | 6.5      | 3.4      | 2.0         | 3.8       | 4.7  |

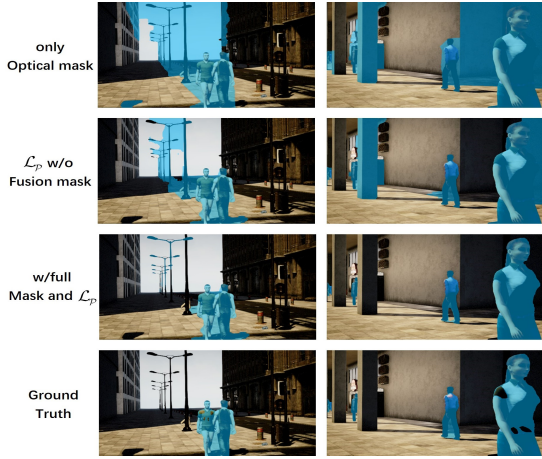


Fig. 3: Evaluation of Tracking Network Loss Methods

the average of five experiments, as shown in Table III. We calculate the average Absolute Trajectory Error (ATE) to assess the impact of each method on the overall system performance. The results in Table IV indicate that all the proposed methods contribute to improved camera tracking. Compared to using only the optical flow estimation strategy, our method improves ATE by 60.52%, indicating that integrating the optical flow mask with the depth mask can effectively enhance pose estimation.

### B. Evaluation of Mask Fusion Accuracy

We compared the mask results obtained with different loss functions  $\mathcal{L}_P$  as shown in Fig. 3. When using only the optical flow mask or only the motion loss similar to DytanVO, significant mask estimation errors appeared. Our strategy achieved the best results, closely approaching the ground truth.

### C. Operating Speed and Network Performance Evaluation

We evaluated the system’s tracking, mapping, and network update speed using the AirDOS-Shibuya dataset as shown in Table IV. In comparison to DytanVO and the state-of-the-art 3DGS-based SLAM system SplaTAM, our approach

demonstrates a superior balance between runtime efficiency and performance. This advantage primarily stems from the absence of a multi-iteration network update strategy in our system.

TABLE III: Ablation study of the proposed method in our systems. The best result is highlighted as **first**.

| Optical Flow | Depth | Optical Flow+Depth | ATE RMSE (cm) ↓ |
|--------------|-------|--------------------|-----------------|
| ✓            | ✗     | ✗                  | 7.6             |
| ✗            | ✓     | ✗                  | 94.8            |
| ✓            | ✓     | ✓                  | <b>3.0</b>      |

TABLE IV: Operating speed and network performance. The best result is highlighted as **first**.

| Method  | Tracking (FPS) ↑ | Mapping (ms) ↑ | Network update (ms) ↓ | GPU memory (MB) ↓ |
|---------|------------------|----------------|-----------------------|-------------------|
| DytanVO | 10.5             | ✗              | 32.9                  | <b>7.6</b>        |
| SplaTAM | 3.8              | 390.4          | ✗                     | 14.6              |
| Ours    | <b>17.0</b>      | <b>430.5</b>   | <b>10.3</b>           | 12.8              |

## VI. CONCLUSIONS

We propose Dy3DGS-SLAM, the first 3DGS-based SLAM method designed for dynamic scenes using monocular RGB input. This method first generates dynamic object masks through optical flow estimation, combining these masks with monocular depth estimation to create a fused mask and recover scale, accurately capturing dynamic object masks. To further improve pose accuracy, we optimized the loss function based on the fused mask, reducing the computational cost associated with multiple iterations. Additionally, to enhance rendering performance, we applied additional photometric and depth losses to eliminate transient interference artifacts and improve geometric accuracy. Experimental results demonstrate that, compared to baseline methods, Dy3DGS-SLAM achieves state-of-the-art tracking and rendering performance in dynamic environments. In the future, we will focus on applying this approach to mobile devices with lower computational costs.

## REFERENCES

- [1] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," *Communications of the ACM*, vol. 65, no. 1, pp. 99–106, 2021.
- [2] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics (TOG)*, vol. 42, no. 4, pp. 139–1, 2023.
- [3] E. Sucar, S. Liu, J. Ortiz, and A. J. Davison, "imap: Implicit mapping and positioning in real-time," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 6229–6238.
- [4] Z. Zhu, S. Peng, V. Larsson, W. Xu, H. Bao, Z. Cui, M. R. Oswald, and M. Pollefeys, "Nice-slam: Neural implicit scalable encoding for slam," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 12786–12796.
- [5] E. Sandström, Y. Li, L. Van Gool, and M. R. Oswald, "Point-slam: Dense neural point cloud-based slam," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 18433–18444.
- [6] F. Tosi, Y. Zhang, Z. Gong, E. Sandström, S. Mattoccia, M. R. Oswald, and M. Poggi, "How nerfs and 3d gaussian splatting are reshaping a survey," *arXiv preprint arXiv:2402.13255*, vol. 4, 2024.
- [7] Y. Zhou, Z. Zeng, A. Chen, X. Zhou, H. Ni, S. Zhang, P. Li, L. Liu, M. Zheng, and X. Chen, "Evaluating modern approaches in 3d scene reconstruction: Nerf vs gaussian-based methods," *arXiv preprint arXiv:2408.04268*, 2024.
- [8] C. Ruan, Q. Zang, K. Zhang, and K. Huang, "Dn-slam: A visual slam with orb features and nerf mapping in dynamic environments," *IEEE Sensors Journal*, vol. 24, no. 4, pp. 5279–5287, 2024.
- [9] M. Li, Y. Zhou, G. Jiang, T. Deng, Y. Wang, and H. Wang, "Ddn-slam: Real-time dense dynamic neural implicit slam with joint semantic encoding," *arXiv preprint arXiv:2401.01545*, 2024.
- [10] Z. Xu, J. Niu, Q. Li, T. Ren, and C. Chen, "Nid-slam: Neural implicit representation-based rgb-d slam in dynamic environments," *arXiv preprint arXiv:2401.01189*, 2024.
- [11] H. Jiang, Y. Xu, K. Li, J. Feng, and L. Zhang, "Rodyn-slam: Robust dynamic dense rgb-d slam with neural radiance fields," *IEEE Robotics and Automation Letters*, vol. 9, no. 9, pp. 7509–7516, 2024.
- [12] N. Keetha, J. Karhade, K. M. Jatavallabhula, G. Yang, S. Scherer, D. Ramanan, and J. Luiten, "Splatam: Splat track & map 3d gaussians for dense rgb-d slam," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 21357–21366.
- [13] H. Huang, L. Li, H. Cheng, and S.-K. Yeung, "Photo-slam: Real-time simultaneous localization and photorealistic mapping for monocular stereo and rgb-d cameras," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 21584–21593.
- [14] H. Matsuki, R. Murai, P. H. Kelly, and A. J. Davison, "Gaussian splatting slam," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 18039–18048.
- [15] Y. Zhu, C. Honnet, Y. Kang, J. Zhu, A. J. Zheng, K. Heinz, G. Tang, L. Musk, M. Wessely, and S. Mueller, "Demonstration of chromocloth: Re-programmable multi-color textures through flexible and portable light source," in *Adjunct Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023, pp. 1–3.
- [16] Y. Song, P. Arora, R. Singh, S. T. Varadharajan, M. Haynes, and T. Starner, "Going blank comfortably: Positioning monocular head-worn displays when they are inactive," in *Proceedings of the ACM International Symposium on Wearable Computers*, 2023, pp. 114–118.
- [17] Y. Song, P. Arora, S. T. Varadharajan, R. Singh, M. Haynes, and T. Starner, "Looking from a different angle: Placing head-worn displays near the nose," in *Proceedings of the Augmented Humans International Conference*, 2024, pp. 28–45.
- [18] C. Yu, Z. Liu, X.-J. Liu, F. Xie, Y. Yang, Q. Wei, and Q. Fei, "Ds-slam: A semantic visual slam towards dynamic environments," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 1168–1174.
- [19] J. He, M. Li, Y. Wang, and H. Wang, "Ovd-slam: An online visual slam for dynamic environments," *IEEE Sensors Journal*, vol. 23, no. 12, pp. 13210–13219, 2023.
- [20] S. Cheng, C. Sun, S. Zhang, and D. Zhang, "Sg-slam: A real-time rgb-d visual slam toward dynamic scenes with semantic and geometric information," *IEEE Transactions on Instrumentation and Measurement*, vol. 72, pp. 1–12, 2022.
- [21] T. Zhang, H. Zhang, Y. Li, Y. Nakamura, and L. Zhang, "Flowfusion: Dynamic dense rgb-d slam based on optical flow," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 7322–7328.
- [22] W. Ye, X. Yu, X. Lan, Y. Ming, J. Li, H. Bao, Z. Cui, and G. Zhang, "Deflow-slam: Self-supervised scene motion decomposition for dynamic dense slam," *arXiv preprint arXiv:2207.08794*, 2022.
- [23] S. Shen, Y. Cai, W. Wang, and S. Scherer, "Dytanvo: Joint refinement of visual odometry and motion segmentation in dynamic environments," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 4048–4055.
- [24] T. Müller, A. Evans, C. Schied, and A. Keller, "Instant neural graphics primitives with a multiresolution hash encoding," *ACM Transactions on Graphics (TOG)*, vol. 41, no. 4, pp. 1–15, 2022.
- [25] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015.
- [26] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao, "Depth anything v2," *arXiv preprint arXiv:2406.09414*, 2024.
- [27] W. Wang, Y. Hu, and S. Scherer, "Tartanvo: A generalizable learning-based vo," in *Conference on Robot Learning*. PMLR, 2021, pp. 1761–1772.
- [28] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [29] G. Kopanas, J. Philip, T. Leimkühler, and G. Drettakis, "Point-based neural rendering with per-view optimization," in *Computer Graphics Forum*, vol. 40, no. 4. Wiley Online Library, 2021, pp. 29–43.
- [30] W. Yifan, F. Serena, S. Wu, C. Öztireli, and O. Sorkine-Hornung, "Differentiable surface splatting for point-based geometry processing," *ACM Transactions on Graphics (TOG)*, vol. 38, no. 6, pp. 1–14, 2019.
- [31] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of rgb-d slam systems," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012, pp. 573–580.
- [32] Y. Qiu, C. Wang, W. Wang, M. Henein, and S. Scherer, "Airdos: Dynamic slam benefits from articulated objects," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022, pp. 8047–8053.
- [33] E. Palazzolo, J. Behley, P. Lottes, P. Giguère, and C. Stachniss, "Refusion: 3d reconstruction in dynamic environments for rgb-d cameras exploiting residuals," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, pp. 7855–7862.
- [34] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. Montiel, and J. D. Tardós, "Orb-slam3: An accurate open-source library for visual, visual-inertial, and multimap slam," *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [35] Z. Teed and J. Deng, "Droid-slam: Deep visual slam for monocular, stereo, and rgb-d cameras," *Advances in Neural Information Processing Systems*, vol. 34, pp. 16558–16569, 2021.
- [36] M. M. Johari, C. Carta, and F. Fleuret, "Eslam: Efficient dense slam system based on hybrid representation of signed distance fields," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 17408–17419.
- [37] H. Wang, J. Wang, and L. Agapito, "Co-slam: Joint coordinate and sparse parametric encodings for neural real-time slam," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 13293–13302.
- [38] B. Bescos, J. M. Fácil, J. Civera, and J. Neira, "Dynaslam: Tracking, mapping, and inpainting in dynamic scenes," *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018.
- [39] C. Kerl, J. Sturm, and D. Cremers, "Dense visual slam for rgb-d cameras," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2013, pp. 2100–2106.